CSC430 – Computer Networks

Fall 2026

Course Project

Live Quiz Arena: Socket Programming

Project Description

In this project, you will design and implement a multiplayer, real-time quiz application inspired by platforms such as Kahoot. The application will use direct **TCP and UDP socket programming** and a **multithreaded client-server architecture**.

A host should be able to create a quiz room, invite players to join, and start a quiz consisting of timed multiple-choice questions. Players submit their answers, receive feedback, and compete for points. The application should display updated scores throughout the quiz and announce the final ranking.

Your task is not only to implement the application, but also to **design its application-layer communication protocol**. You must decide how clients and the server exchange information, which messages require reliable delivery, which can tolerate occasional loss, and how the application maintains consistent state despite concurrent operations and network errors.

Functional Requirements

Your application must provide the following functionality.

1. **Room creation and joining.** A host can create a quiz room and obtain a unique room identifier (e.g., a room PIN). Players can join an existing room using its identifier and a chosen username. The application must reject invalid room identifiers and duplicate usernames within the same room.

2. **Quiz initialization.** The host can start the quiz after players have joined. Questions should be loaded from a file or another simple data source. Each question must have multiple choices and one correct answer.

3. **Timed questions.** Questions are presented to players with a time limit. Players should be informed of the remaining time, either through periodic updates or another mechanism of your design. The server must determine when a question opens and closes.

4. **Answer submission.** Players can submit an answer to the current question. The server must validate submitted answers and reject invalid, duplicate, or late submissions. Each player may submit at most one accepted answer per question.

5. **Scoring and leaderboards.** The server maintains player scores and determines the correctness of answers. You may design your own scoring policy, provided it is clearly documented and applied consistently. Players should receive updated scores or rankings after each question.

6. **Quiz completion.** After the final question, the application displays the final leaderboard, including all participating players and their scores.

7. **Multiple rooms.** A single server must support multiple independent quiz rooms simultaneously. Each room must maintain its own players, questions, timers, and scores. At minimum, demonstrate three concurrent rooms with at least five players each.

8. **Client interface.** A terminal-based interface is sufficient. A graphical interface is optional and does not earn additional credit by itself.

Technical Requirements

1. Client-Server Architecture

Your application must include a server program and a client program, implemented **in Python**.

The server must remain running and continue accepting connections until manually stopped. Clients should be able to exchange multiple messages during a session without repeatedly restarting the application.

The server is responsible for maintaining the authoritative quiz state, including room membership, question timing, answer acceptance, and scoring.

2. TCP and UDP Communication

Both TCP and UDP socket programming are mandatory.

Your application must use both protocols for meaningful parts of its functionality. You must decide which categories of messages should be transmitted using TCP and which should use UDP. You must **justify your choice of transport protocol for each message category** in your report.

3. Application-Layer Protocol Design

You must design your own application-layer protocol for communication between clients and the server. Your protocol specification must define:

- All supported message types and their purposes.
- The direction of each message (client-to-server or server-to-client).
- The transport protocol used for each message type (TCP or UDP).
- The format and fields of each message.
- The expected sequence/order of messages during the main application operations.
- How invalid or unexpected messages are handled.

You are free to use JSON, a structured text format, or another clearly documented encoding.

Message boundaries: TCP provides a byte stream rather than individual messages. Your protocol must therefore define a mechanism for identifying complete messages over TCP.

Your implementation must correctly handle cases where a single TCP read contains only part of a message or multiple messages.

For UDP, you must account for its datagram-oriented behavior and possible message loss, duplication, or reordering.

4. Multithreading and Concurrency

Multithreading is required.

Your application must use multiple threads to support concurrent activities, such as serving multiple clients, managing rooms, receiving messages, or handling quiz timers.

You are free to choose the organization of threads, but you must explain your design. Since several threads may access shared data, you must identify and correctly handle potential concurrency problems, including race conditions, deadlocks, or inconsistent shared states. To this end, use appropriate synchronization mechanisms (e.g., locks or other thread-safe techniques) where needed. Your solution should preserve correctness under concurrent operations, rather than relying on the assumption that conflicting operations will rarely occur.

5. Error Handling and Robustness

Your application must handle errors gracefully, without unnecessarily terminating the server or disrupting unrelated clients and rooms. For example, consider the following cases:

- Invalid input or malformed messages.
- Requests that are not valid in the current application state.
- Unexpected client disconnections.
- Socket errors and connection failures.
- Lost, duplicated, or out-of-order UDP datagrams.
- Concurrent operations that could produce inconsistent results.

You must define reasonable behavior for these situations and document your decisions.

In particular, consider how your application can remain consistent if an important UDP message is lost. You may implement recovery, retransmission, state synchronization, or another suitable mechanism.

Not every lost message needs to be recovered, but your design must explain which losses can be tolerated and which require action.

6. Network Connectivity

Your application must work over a network, not only through communication between processes on the same computer. Demonstrate successful communication with at least one client running on a different device from the server. You should also demonstrate multiple clients participating in a quiz simultaneously.

You may assume that the devices are on a reachable local network. NAT traversal and Internet deployment are not required.

7. Implementation Constraints

- Use Python and standard socket programming libraries.
- You may use standard libraries for threading, synchronization, serialization, and timing.
- **Do <u>not</u> use frameworks that abstract away TCP/UDP socket communication, such as WebSocket, Socket.IO, or HTTP-based networking frameworks.**
- A database is not required. Simple files (JSON, CSV, or text) are sufficient for storing quiz questions.
- Keep your implementation reasonably modular, readable, and documented.

Deliverables

1. Application, Source, and Documentation Files

Submit your application and all source code files required to run your application, including the server, client, and any supporting files or test scripts (if any). Your code must be properly documented:

- At the top of each Python file, describe its purpose.
- Document the purpose, inputs, and outputs of important functions.
- Explain important implementation decisions, especially socket operations, message parsing, thread synchronization, and error handling.

Include clear instructions for launching the server and clients (a README file) including any required configuration or dependencies.

2. Technical Report

Submit a technical report in **PDF format** containing the following sections.

1. **Application Description.** Briefly describe the application and its main functionality.
2. **Application-Layer Protocol.** Document the message types, formats, fields, and expected exchanges. Include a table identifying the TCP or UDP transport used for each message type, with justification.

   Explain how message boundaries are determined and how your protocol handles missing, invalid, or unexpected messages.

   Include at least one sequence diagram illustrating a typical quiz session.
3. **Design and Implementation.** Explain the client-server architecture, the organization of threads, and the management of rooms and quiz state.

   Identify shared resources and explain how synchronization prevents race conditions and inconsistent updates.

   Describe your timing and scoring policies and other important design decisions.
4. **Testing and Sample Runs.** Include screenshots showing the application's main operations and different test scenarios.

   Show how the server IP address was obtained and used by clients on different devices, and demonstrate how multiple multiplayer rooms can operate concurrently. You may write testing scripts to simulate multiple players and rooms if sufficient actual players are not available for testing.

   Include Wireshark captures for TCP and UDP (refer to the Wireshark tutorial and sample lab provided on Blackboard), and describe the results of your concurrency, error-handling, and other tests.
5. **Challenges and Limitations.** Discuss implementation challenges, unresolved issues, and limitations. Explain any debugging attempts and improvements made during development.
6. **References.** List all resources used, including documentation, tutorials, and AI tools such as ChatGPT, if applicable. External resources should support learning and understanding rather than blind copying.

3. Live Demonstration

Each team must provide a live demonstration of the completed project. The demonstration should show the main required functionality of the application and may include testing scripts to simulate additional players or concurrent rooms when needed. During the demonstration, all team members should be prepared to answer questions about the design and implementation of the project, including the application-layer protocol, the use of TCP and UDP, multithreading, synchronization, and error handling. The live demonstration will also be used to assess each team member's individual understanding and contribution.

Note:

- Members of the same team do not necessarily receive the same grade.
- The live demonstration counts for 50% of the overall project grade.

Evaluation Criteria

Your project will be evaluated based on:

- Correctness and completeness of the required quiz functionality.
- Quality and clarity of the application-layer protocol design.
- Appropriate and justified use of both TCP and UDP.
- Correct multithreading and synchronization of shared state.
- Robustness and graceful handling of network and application errors.
- Quality of testing and evidence of correct network communication.
- Code organization, documentation, and quality of the report.

Final Notes

This project is intended to be achievable within approximately one month. The main objective is to demonstrate your understanding of **application-layer protocol design, TCP and UDP socket programming, multithreading, and robust client-server communication**.

A correct, well-designed application with a clearly documented protocol is preferable to an overly ambitious application with many incomplete features. You are encouraged to make thoughtful design decisions and to justify them based on the networking concepts studied in class.